

Informe de Revisión Técnica Cruzada del Código del PFC

FORM-TL-05 — Principios SOLID, patrones de diseño, separación de capas,
manejo de excepciones distribuidas y observabilidad

Sistema auditado: **SGA – Sistema de Gestión Académica Distribuido**
(equipo autor **BCEL**, sigla institucional “Escuela Provincias Unidas”)

Equipo revisor: **AGLS**

(PFC propio del equipo revisor: TiendaTech – comercio electrónico, no auditado en este informe)

Período 2026–2027 · Semana 16

Datos del equipo REVISOR (AGLS)

Integrante 1 – Rol	Jhinson Stalyn Aucatoma Celorio – Bloques A, B y C (SOLID, patrones y separación de capas)
Integrante 2 – Rol	Andy Paul Sánchez Pilaloe – Bloques D y E (excepciones distribuidas y trazabilidad)

Datos del PFC REVISADO (BCEL)

Equipo revisado (sigla)	BCEL
Nombre del sistema	SGA – Sistema de Gestión Académica Distribuido (Escuela Provincias Unidas)
URL del repositorio revisado	https://github.com/LE023as/sga-sistema-distribuido
Commit auditado (SHA corto)	ff6f8b0 (completo: ff6f8b0bf3f4200ae4deb9d6d7833e8bce7330f4)
Fecha y hora de la auditoría	11 de agosto de 2026, 08:30

Nota de verificación del objeto revisado

Siguiendo la práctica profesional de revisión de código moderna, cuyo valor documentado excede la sola detección de defectos y se centra en la transferencia de conocimiento entre equipos [1], [2], el análisis inicial se elaboró sobre el archivo **sga-sistema-distribuido-main.zip** entregado como material de trabajo, correspondiente a una descarga “Download ZIP” de la rama **main** y sin directorio **.git**. Posteriormente el equipo AGLS completó la verificación exigida en la Sección 6 de la guía FORM-TL-05 sobre un clon real del repositorio:

```
git clone https://github.com/LE023as/sga-sistema-distribuido.git revision-pfc
cd revision-pfc
git log -1 --format="%H | %aI | %an"
git rev-parse --short HEAD
git ls-files | wc -l
```

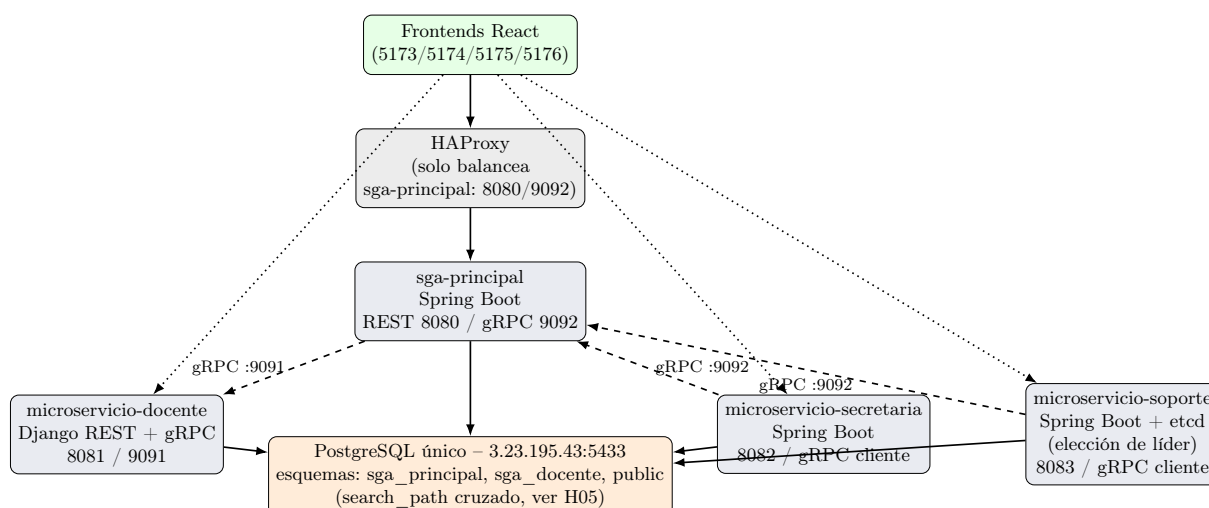
confirmando como commit auditado el **ff6f8b0** (SHA completo **ff6f8b0bf3f4200ae4deb9d6d7833e8bce7330f4**), verificado el 11 de agosto de 2026 a las 08:30, valores ya reflejados en la portada. Todos los hallazgos de este informe referencian rutas de archivo estables dentro del árbol de trabajo (**sga-principal/**, **microservicio-docente/**, **microservicio-secretaria/**, **microservicio-soporte/**), por lo que siguen siendo válidos independientemente de cambios posteriores al commit auditado, siempre que se referencien contra ese SHA exacto.

Índice

1. Mapa de arquitectura reconstruido	1
2. Tabla consolidada de hallazgos	1
3. Análisis por bloque	3
3.1. Bloque A – Principios SOLID	3
3.2. Bloque B – Patrones de diseño y antipatronos	4
3.3. Bloque C – Separación de capas	5
3.4. Bloque D – Manejo de excepciones distribuidas	5
3.5. Bloque E – Logging y trazabilidad	5
4. Tabla de issues abiertos	6
5. Preguntas de análisis obligatorias	7
6. Conclusiones y recomendación priorizada	8

1 Mapa de arquitectura reconstruido

El sistema BCEL se compone de un servicio núcleo (**sga-principal**, Java 17 / Spring Boot) y tres microservicios autónomos (**microservicio-docente** en Django REST + gRPC, y **microservicio-secretaria** / **microservicio-soporte**, ambos Spring Boot pese a que el `README.md` los documenta como “Node.js/Express”¹). La reconstrucción se hizo a partir de los `application.properties/settings.py`, los clientes gRPC y `docker-compose.yml`, no de la documentación.



Flechas sólidas: REST/HTTP. Flechas discontinuas: gRPC (PLAINTEXT, sin TLS). Flechas punteadas: acceso directo de cada frontend a su propio backend, sin pasar por HAProxy (HAProxy sólo tiene `frontend/backend` definidos para **sga-principal**, ver `infra/haproxy/haproxy.cfg` líneas 24–47). **sga-principal** actúa de facto como *hub* gRPC: **secretaria** y **soporte** lo consultan para el catálogo institucional (años, grados, estudiantes), y él a su vez consulta a **docente** para actividades/asistencias. **soporte** incorpora además un algoritmo propio de elección de líder sobre `etcd` (`election/LeaderElectionService.java`), un acierto de diseño distribuido que se reconoce en la Sección 6.

2 Tabla consolidada de hallazgos

Cada hallazgo se clasifica además por el atributo de calidad de producto que compromete, siguiendo la nomenclatura de ISO/IEC 25010 [3] (mantenibilidad, fiabilidad, seguridad o eficiencia de desempeño, según el caso); esa clasificación se detalla en el cuerpo de cada issue del Anexo A (archivo `issues_plan.md`) para no duplicar columnas en esta tabla. Cada hallazgo referencia archivo y línea dentro del árbol de trabajo del repositorio revisado. La columna *Item* usa la nomenclatura de la Sección 5 de la guía FORM-TL-05 (bloques A–E). La columna *Issue* enlaza al número que deberá asignarse al abrir el issue en el repositorio de BCEL (Sección 4).

ID	Item	Hallazgo	Evidencia (ruta:línea)	(ru- Sever.	#
H01	A1	<code>ImportacionExcelService</code> concentra parseo de CSV, Excel y PDF y la persistencia de estudiantes/matrículas en una sola clase de 388 líneas.	<code>sga-principal/.../servi ce/ImportacionExcelSer vice.java:43-388</code>	Mayor	1

¹`README.md`, tabla de arquitectura: declara Node.js/Express para Secretaría y Soporte; el código real en `microservicio-secretaria/backend/pom.xml` y `microservicio-soporte/backend/pom.xml` es Maven/Spring Boot. Documentación desactualizada, ver Anexo de sugerencias.

ID	Item	Hallazgo	Evidencia (ruta:línea)	(ru- Sever.	#
H02	A2	Agregar un formato de archivo nuevo (p.ej. .ods) obliga a editar el <code>if/else</code> existente en <code>parsearArchivo</code> y añadir un método <code>leerX</code> paralelo.	.../ImportacionExcelService.java:61-70	Menor	2
H03	A5/C2	<code>AuthController</code> inyecta <code>UsuarioRepository</code> directamente y lo consulta con <code>.orElseThrow()</code> sin manejar, saltando la capa de servicio.	.../controller/AuthController.java:9,23,39-41	Mayor	3
H04	A1/A5/C2	<code>PrincipalGrpcService</code> (adaptador gRPC, capa de comunicación) inyecta 4 repositorios además de 2 servicios y mezcla marshalling, acceso a datos y reglas de 5 agregados distintos en 382 líneas.	.../grpc/PrincipalGrpcService.java:37-44	Mayor	4
H05	B5/C5	El esquema de <code>microservicio-docente</code> declara <code>search_path=sga_docente,sga_principal,public:</code> puede leer/escribir directamente el esquema de otro servicio. Los 4 backends además convergen en la misma instancia física de PostgreSQL.	micro_docente/settings.py:53-65 (l.63)	Crítica	5
H06	C5	Contraseña de BD, secreto JWT, contraseña de aplicación de Gmail y token interno de gRPC hardcoded y versionados en texto claro; credenciales reales de acceso publicadas en el README.md.	application.properties:10,29,33-34; settings.py:58; InternalAuthInterceptor.java:15; README.md	Crítica	6
H07	D2	Ningún cliente gRPC del sistema (<code>Actividad/Asistencia/DocenteGrpcClient</code> , <code>PrincipalGrpcClient</code> , <code>TecnicoGrpcClient</code>) fija <code>withDeadlineAfter</code> ; las llamadas son bloqueantes sin límite de espera propio.	.../grpc/ActividadGrpcClient.java:30-37 (5 clientes)	Mayor	7
H08	D3/D4	No existe política de reintentos ni respuesta de degradación: los <code>catch</code> sólo relanzan la excepción original o la traducen 1:1 a un error HTTP, sin valor de reserva.	ActividadGrpcClient.java:33-36,42-45,52-55,67-70,77-80	Mayor	7
H09	D5	Formato de error no uniforme entre servicios: <code>secretaria</code> y <code>soporte</code> tienen <code>@RestControllerAdvice</code> propio; <code>sga-principal</code> no tiene ningún manejador global y depende de <code>ResponseStatusException</code> ad-hoc por método.	sga-principal: sin <code>@RestControllerAdvice</code> (0 result.) vs. .../soporte/.../GlobalExceptionHandler.java:1-127	Mayor	8
H10	E1	Registro por consola (<code>System.out/err.println</code>) en vez de logger inyectado, incluyendo el punto de entrada de la aplicación.	DocenteContextController.java:31-86; SgaPrincipalApplication.java:14; DataSeedRunner.java:47-182; ActividadGrpcClient.java:34,43,53,68,78	Menor	9
H11	E2	No existe identificador de correlación en ningún punto del sistema (0 coincidencias de <code>MDC/correlationId/traceId/X-Request-Id</code> en los 3 backends Java ni en <code>settings.py</code> de Django).	Búsqueda global <code>grep -rln</code> en los 4 servicios (0 result.)	Mayor	10
H12	E4	<code>main()</code> imprime en cada arranque el hash bcrypt de una contraseña hardcoded; el README.md publica usuario y contraseña reales de administrador y docente en texto claro.	SgaPrincipalApplication.java:12-16; README.md (§ Credenciales)	Crítica	11

ID	Item	Hallazgo	Evidencia (ruta:línea)	(ru- Sever.	#
H13	B4	La única puerta de enlace (HAProxy) balancea exclusivamente <code>sga-principal</code> ; <code>docente</code> , <code>secretaria</code> y <code>soporte</code> no tienen backend definido, por lo que el frontend debe conocer y llamar cada puerto de servicio directamente.	<code>infra/haproxy/haproxy.cfg:24-47</code>	Menor	12
H14	Sug.	Higiene de repositorio: clase pública vacía con nombre en minúscula (<code>repository.java</code>); binarios <code>protoc-*.exe</code> versionados en <code>target/</code> pese a que <code>.gitignore</code> excluye <code>**/target/</code> ; <code>.gitignore</code> con codificación corrupta en sus últimas líneas.	<code>.../repository/repository.java;</code> <code>.../soporte/backend/target/protoc-plugins/;</code> <code>.gitignore</code>	Sug.	13

3 Análisis por bloque

3.1 Bloque A – Principios SOLID

Los cinco principios se verifican en el sentido estructural definido por Martin [4]: producir módulos que toleren el cambio, no una preferencia de estilo.

SRP (violado, H01/H04). `ImportacionExcelService` resuelve tres responsabilidades independientes – parseo de CSV, parseo de Excel (Apache POI) y extracción heurística de listados PDF con expresiones regulares (Apache PDFBox) – y además ejecuta la persistencia final de `Estudiante` y `Matricula` dentro del mismo método `confirmarImportacion` (líneas 319-387). El propio código documenta esta deuda en su Javadoc de clase (líneas 34-39: “mismo bypass que `ImportacionCasService`... escribe estudiantes/matriculas por JPA directo, sin pasar por `sga-secretaria`”), lo que confirma que el equipo autor es consciente del atajo pero no lo ha encapsulado. El caso más costoso es `PrincipalGrpcService` (H04): al ser el único punto de entrada gRPC que consultan *dos* microservicios distintos (`secretaria` y `soporte`), cualquier cambio en su forma de acceder a datos tiene radio de impacto en todo el sistema distribuido, no sólo en `sga-principal`.

OCF (violado, H02). `parsearArchivo` decide el parser por medio de una cadena `if/else if` sobre la extensión del nombre de archivo (líneas 61-70). Cada formato nuevo exige tocar este método y añadir un `leerX` paralelo; no hay una abstracción (`interface ImportadorArchivo`) que permita registrar un nuevo formato sin modificar código existente.

Listing 1: Cadena condicional cerrada a extensión, `ImportacionExcelService.java:61-70`

```

1 if (nombreArchivo.endsWith(".csv")) {
2     filas = leerCsv(archivo);
3 } else if (nombreArchivo.endsWith(".xlsx") || nombreArchivo.endsWith(".xls")) {
4     filas = leerExcel(archivo);
5 } else if (nombreArchivo.endsWith(".pdf")) {
6     filas = leerPdf(archivo);
7 } else {
8     throw new ResponseStatusException(HttpStatus.BAD_REQUEST, ...);
9 }

```

LSP e ISP (sin violación exigible – NA justificado). Se buscó de forma explícita jerarquías de clases e interfaces propias del dominio (`grep "class .* extends"` en los tres backends Java). El único hallazgo son las clases generadas de gRPC (`PrincipalServiceGrpc.PrincipalServiceImplBase`) y el filtro estándar de Spring Security (`OncePerRequestFilter`); ambas se implementan sin estrechar precondiciones ni lanzar `UnsupportedOperationException`. El sistema tampoco define interfaces de servicio propias (los `@Service` son clases concretas inyectadas directamente), por lo que no existe superficie donde ISP pueda violarse. Marcar estos dos ítems como “no aplica” es más honesto que forzar un hallazgo:

la ausencia de abstracciones no es en sí misma una violación de LSP/ISP, pero sí es la causa raíz del siguiente punto.

DIP (violado, H03/H04). La ausencia casi total de interfaces de servicio explica por qué las capas superiores terminan dependiendo de implementaciones concretas de infraestructura en vez de abstracciones. `AuthController` (capa de presentación) inyecta `UsuarioRepository` (capa de persistencia) en lugar de pasar exclusivamente por `UsuarioService`:

Listing 2: `AuthController.java:9,23,39-41` – controlador dependiendo de un repositorio concreto

```
1 import ec.edu.uteq.sga.repository.UsuarioRepository;
2 ...
3 private final UsuarioRepository usuarioRepository;
4 ...
5 Usuario usuario = usuarioRepository.findByUsername(request.getUsername())
6     .orElseThrow();
```

Del lado de `PrincipalGrpcService`, la capa de comunicación remota importa cuatro repositorios (`AnoLectivoRepository`, `AsignaturaRepository`, `RepresentanteRepository`, `EstudianteRepository`) en paralelo a dos servicios (`EstudianteService`, `GradoService`), de modo que algunas operaciones del mismo agregado pasan por el servicio y otras acceden al repositorio directamente: no hay un contrato único y estable que las demás capas puedan sustituir o mockear.

3.2 Bloque B – Patrones de diseño y antipatrones

El catálogo de referencia usado para clasificar patrones y antipatrones es el clásico de Gamma et al. [5] para los primeros, y los catálogos específicos de arquitecturas de microservicios para los segundos [6], [7], [8].

Patrones bien aplicados. El uso de *DTO* (Data Transfer Object) es consistente y correcto en `sga-principal` y, sobre todo, en `microservicio-secretaria` (paquete `dto/` con más de 15 clases `Request/Response` independientes de las entidades JPA). El patrón *Repository* vía Spring Data JPA se respeta en la mayoría del código (interfaces `XxxRepository extends JpaRepository`, sin SQL crudo expuesto al controlador). El manejador centralizado de excepciones de `secretaria/soporte` (`@RestControllerAdvice`, Sección 3 Bloque D) es una aplicación correcta del patrón *Chain of Responsibility / Interceptor* para normalizar errores.

Patrón mal aplicado (H04). `PrincipalGrpcService` se presenta como un adaptador/gateway gRPC, pero al inyectar repositorios directamente se comporta como un *Repository* disfrazado de *Facade*: expone operaciones de bajo nivel de cuatro agregados distintos sin que ese acoplamiento quede documentado en ningún contrato (`.proto` sólo describe la forma del mensaje, no quién puede tocar qué tabla).

Patrones distribuidos (H13, parcial). Siguiendo los patrones de descomposición y comunicación descritos por Richardson [9] y Newman [10], existe *Load Balancer* con *health check* activo (HAProxy, `httpchk GET /actuator/health/liveness, fall 2 rise 2`), pero sólo delante de `sga-principal`; no hay una *API Gateway* real que unifique el enrutamiento de los cuatro servicios bajo un mismo punto de entrada, ni *Circuit Breaker* ni *Saga* en ninguna transacción que cruce servicios (p. ej. crear matrícula en `secretaria` + registrar en `principal` vía gRPC no tiene compensación documentada si el segundo paso falla).

Antipatrón de base de datos compartida (H05, crítico). `microservicio-docente/micro_docente/settings.py` configura explícitamente el `search_path` de su conexión con acceso al esquema de otro servicio:

Listing 3: `micro_docente/settings.py:53-65`

```
1 DATABASES = {
2     "default": {
3         "ENGINE": "django.db.backends.postgresql",
```

```

4      ...
5      "HOST": os.environ.get("DB_HOST", "3.23.195.43"),
6      "PORT": os.environ.get("DB_PORT", "5433"),
7      "OPTIONS": {
8          "options": "-c search_path=sga_docente,sga_principal,public",
9      },
10     },
11 }

```

Tal como advierten los estudios de reingeniería hacia microservicios, este tipo de decisión debe evaluarse con criterios explícitos de acoplamiento y cohesión entre servicios, y no con la impresión subjetiva del revisor [11]. Esto no es un desliz de una única consulta: es la configuración de conexión por defecto, por lo que *cualquier* consulta ORM de **docente** puede alcanzar tablas de **sga_principal** sin pasar por el gRPC que el propio equipo diseñó para ese propósito. Sumado a que los cuatro backends apuntan al mismo host y puerto físico (3.23.195.43:5433), el sistema no cumple *database per service* pese a estar descrito como “persistencia de datos distribuida” en el README.md.

3.3 Bloque C – Separación de capas

La estructura de paquetes (**controller/service/repository/dto/entity**) existe y en su mayoría se respeta (C1, cumple). La dirección de dependencias hacia el dominio, sin embargo, se rompe en dos puntos ya evidenciados en el Bloque A: **AuthController** y **PrincipalGrpcService** importan clases de **repository** sin pasar por **service** (C2, no cumple). Las entidades JPA no se filtran directamente en las respuestas REST de **sga-principal** ni de **secretaria/sopORTE** (C3, cumple: se usan **DTO Response** en todos los controladores revisados). La configuración sensible **no** está externalizada de forma segura (C5, no cumple, ver H06): aunque se usa el patrón **\$VAR:default** de Spring, el valor por defecto embebido es un secreto real y funcional, no un valor de ejemplo, por lo que basta con *no* definir la variable de entorno para que el sistema arranque con la contraseña de producción tal como quedó versionada.

3.4 Bloque D – Manejo de excepciones distribuidas

No se encontraron bloques **catch** que descarten la excepción en silencio (D1, cumple): toda captura relanza o registra, aunque una parte lo hace por **System.err.println** en vez de un logger (ver Bloque E). El punto más débil del sistema es la ausencia total de **withDeadlineAfter/Deadline** en los cinco clientes gRPC (D2, no cumple, H07): una llamada bloqueante sin límite de tiempo propio depende únicamente de la configuración por defecto del canal, que tampoco se fija explícitamente. No existen reintentos en ningún cliente (D3, no aplica por ausencia total – no hay política que evaluar), y por la misma razón tampoco existe degradación ni valor de reserva ante un servicio caído (D4, no cumple, H08): el **catch (StatusRuntimeException)** de **ActividadGrpcClient** sólo hace **System.err.println** y relanza la excepción original hacia el controlador, que no la traduce a un mensaje de negocio. El formato de error expuesto al cliente tampoco es uniforme entre servicios (D5, no cumple, H09): **secretaria** y **sopORTE** devuelven **{"error": ..., "detalle": ...}** vía **GlobalExceptionHandler**, mientras que **sga-principal** no tiene ese manejador y deja que Spring Boot devuelva su cuerpo de error por defecto ante cualquier excepción no cubierta por un **ResponseStatusException** explícito (por ejemplo, el **NoSuchElementException** de **AuthController.java:41**).

3.5 Bloque E – Logging y trazabilidad

Del código de **sga-principal** sólo cuatro clases usan **@Slf4j** (**EmailService**, **ImportacionCasService**, **ImportacionExcelService**, **UsuarioService**); el resto de puntos de depuración usa **System.out/err.println**, incluido el propio **main()** (E1, no cumple, H10). No

existe ningún identificador de correlación: los metadatos gRPC que sí se propagan (`docente_id`, `internal_token`) autentican al llamante pero no identifican la petición, por lo que no sirven para unir registros de distintos servicios (E2, no cumple, H11). Al no existir E2, tampoco puede cumplirse un registro estructurado filtrable por identificador de correlación (E3, no cumple por dependencia directa de E2), aunque los mensajes que sí pasan por Slf4j usan parámetros con `{}` en vez de concatenación de cadenas, lo cual es una base aprovechable para migrar a JSON estructurado. Se encontró exposición de datos sensibles (E4, no cumple, crítico, H12): el hash de una contraseña hardcodeda se imprime en cada arranque de `sga-principal`, y el `README.md` publica usuario y contraseña reales de dos roles. Los eventos críticos del dominio (matrícula, creación de ticket) sí generan al menos un registro de persistencia auditable en base de datos vía los campos `fecha_creacion/creado_por` de las entidades (E5, cumple parcialmente), pero sin acompañamiento de un registro de aplicación correlacionable.

4 Tabla de issues abiertos

Estado de apertura

Los 12 issues listados abajo (GitHub #2–#9 y #11–#14) ya fueron abiertos en el repositorio de BCEL con el contenido completo del Anexo A (título, contexto, evidencia, diagnóstico, propuesta de corrección, criterio de aceptación); el archivo `issues_plan.md` entregado junto a este informe conserva el cuerpo de cada uno. La columna *Resp.* refleja la asignación propuesta por AGLS por bloque temático (Bedón Viteri: Bloque A/SOLID; Castro López: BD compartida, secretos y resiliencia gRPC; Emanuel Pino: manejo de errores y trazabilidad; Luna Mora: exposición de credenciales, gateway y limpieza de repositorio), pendiente de reflejarse todavía como *assignee* real en GitHub. Al momento de esta verificación ningún issue tiene todavía *label* ni *assignee* asignado en GitHub: falta que el equipo BCEL aplique las etiquetas de severidad/bloque y se autoasigne según esta propuesta. Además, el issue #10 quedó creado como duplicado exacto del #9 (mismo título “Unificar el formato de error con un `@ControllerAdvice` en `sga-principal`”), y el issue de higiene de repositorio (Sugerencia, hallazgo H14) **no llegó a crearse**: se recomienda editar el #10 para convertirlo en ese issue de higiene (en vez de cerrarlo como duplicado y perder el número) usando el cuerpo correspondiente de `issues_plan.md`.

#	Título	Bloque / Sever.	Enlace	Resp.	Fecha límite
1	Extraer el parseo de archivos de <code>ImportacionExcelService</code> en estrategias por formato (SRP)	A1 / Mayor	#2	Bedón Viteri	Cierre S17
2	Sustituir el <code>if/else</code> de <code>parsearArchivo</code> por un registro de <code>ImportadorArchivo</code> (OCP)	A2 / Menor	#3	Bedón Viteri	Cierre S17
3	Enrutar <code>AuthController</code> exclusivamente a través de <code>UsuarioService</code> (DIP/capas)	A5/C2 / Mayor	#4	Bedón Viteri	Cierre S17
4	Dividir <code>PrincipalGrpcService</code> en adaptadores por agregado que sólo llamen a servicios (SRP/DIP)	A1/A5/C2 / Mayor	#5	Bedón Viteri	Cierre S17
5	Aislar el esquema de <code>microservicio-docente</code> (rol de BD sin acceso a <code>sga-principal</code>)	B5/C5 / Crítica	#6	Castro López	Cierre S17

#	Título	Bloque / Sever.	Enlace	Resp.	Fecha límite
6	Externalizar y rotar todos los secretos versionados (BD, JWT, SMTP, token gRPC)	C5 / Crítica	#7	Castro López	Cierre S17
7	Añadir <code>deadline</code> , reintento acotado y degradación a los clientes gRPC	D2/D3/D4 / Mayor	#8	Castro López	Cierre S17
8	Unificar el formato de error con <code>@ControllerAdvice</code> en <code>sga-principal</code>	D5 / Mayor	#9	Emanuel Pino	Cierre S17
9	Reemplazar <code>System.out/err.println</code> por el logger inyectado en toda la base	E1 / Menor	#11	Emanuel Pino	Cierre S17
10	Propagar un identificador de correlación entre HTTP y gRPC	E2 / Mayor	#12	Emanuel Pino	Cierre S17
11	Eliminar la impresión del hash de contraseña en <code>main()</code> y las credenciales del <code>README.md</code>	E4 / Crítica	#13	Luna Mora	Cierre S17
12	Ampliar HAProxy (o introducir un API Gateway) para los tres microservicios restantes	B4 / Menor	#14	Luna Mora	Cierre S17
13	Limpieza de repositorio: clase vacía, binarios de <code>target/</code> versionados, <code>.gitignore</code> corrupto	Sugerencia	#10 (pendiente: reutilizar)	Luna Mora	Cierre S17

Referencia de columnas truncadas

La columna *Enlace* apunta al número real asignado por GitHub a cada issue (verificado programáticamente contra la API pública del repositorio). La columna *Resp.* muestra el integrante de BCEL propuesto por AGLS para cada issue (Keyla Bedón Viteri, Pedro Castro López, Juliana Emanuel Pino y Ernesto Luna Mora); a la fecha de esta verificación esa asignación todavía no se refleja como *assignee* de GitHub en ningún issue – el equipo BCEL debe autoasignarse cada uno según esta propuesta (o redistribuirla si lo prefieren). “Cierre S17” significa: cualquier fecha anterior al cierre de la Semana 17 del calendario del curso.

5 Preguntas de análisis obligatorias

- P1. Qué violación SOLID encarece más el cambio de cara a la Entrega 4.** De las cuatro violaciones documentadas, la que más encarece el cambio futuro es la de `PrincipalGrpcService` (H04, SRP+DIP), no la de `ImportacionExcelService` (H01) a pesar de tener más líneas. La razón es el radio de impacto: `ImportacionExcelService` sólo la usa `sga-principal` en un flujo interno de carga masiva; `PrincipalGrpcService` es el único punto de acceso remoto que consultan *dos* microservicios externos (`secretaria` y `soporte`). Cualquier refactor de sus repositorios internos, o cualquier bug introducido al añadir un nuevo método, se propaga inmediatamente a dos equipos de desarrollo distintos que dependen del mismo `.proto` y del mismo comportamiento implícito (qué endpoint pasa por el servicio y cuál accede al repositorio directo no está documentado en ninguna parte).
- P2. Patrón de diseño a introducir y por qué una solución más simple no basta.** Se introduciría *Circuit Breaker* (por ejemplo con Resilience4j) envolviendo los cinco clientes gRPC identificados en H07/H08. El problema concreto que resuelve es el de fallo parcial no contenido: hoy, si `microservicio-docente` deja de responder, cada petición de `sga-principal` hacia él sigue intentándolo sin límite de tiempo propio (no hay `deadline`) y sin abrir un circuito que corte los intentos tras detectar fallos repetidos. Un simple

`try/catch` con reintento manual no basta porque no tiene memoria entre peticiones: cada nueva petición de un usuario distinto repite el mismo costo de espera y el mismo fallo, agotando hilos del servidor uno a uno hasta el colapso; el Circuit Breaker sí recuerda el estado del servicio remoto y falla rápido para las peticiones siguientes mientras el servicio no se recupera.

- P3. Qué ocurre si un servicio dependiente deja de responder 30 segundos.** Trazando el camino de, por ejemplo, `DocenteActividadController` → `ActividadGrpcClient.crearActividad()` (`ActividadGrpcClient.java:30-37`): el método llama a `getStubWithMetadata().crearActividad(request)` sobre un `BlockingStub` sin `withDeadlineAfter`. Si `microservicio-docente` deja de responder (proceso vivo pero colgado, no caído), el hilo HTTP de Tomcat que atiende esa petición de `sga-principal` queda bloqueado durante los 30 segundos completos – y más, porque no hay límite propio – sin liberar el hilo del pool. El punto de fallo exacto es la ausencia de `Deadline` en esa línea: si varias peticiones concurrentes golpean el mismo endpoint durante la ventana de indisponibilidad, el pool de hilos de `sga-principal` se agota y el servicio deja de responder también a peticiones que no tenían nada que ver con `docente` (por ejemplo, un simple `GET /api/estudiantes`), es decir, se produce una falla en cascada.
- P4. Es posible hoy reconstruir la traza completa de una operación multi-servicio.** No. Como se documenta en E2 (H11), no existe ningún identificador de correlación generado en el borde del sistema ni propagado entre `sga-principal` y `microservicio-docente`. Un ejemplo real: al registrar una actividad, `DocenteContextController` imprime por consola líneas como "[API DEBUG] /mis-asignaciones: Docente obtenido ->ID Persona: 12, Nombre: ..." (`DocenteContextController.java:33`), pero ese texto no lleva ningún campo que también aparezca en los registros que genera `microservicio-docente` para la misma petición. Un ingeniero que investigue un incidente sólo puede intentar correlacionar por proximidad de marca de tiempo, lo cual es poco fiable bajo carga concurrente y no es reproducible como evidencia.
- P5. Qué aprendizaje de arquitectura se aplicará al PFC propio (TiendaTech) antes de la Entrega 4.** Cuatro decisiones concretas: (1) usar DTO de entrada/salida desde el primer commit – `secretaria` y `soporte` lo hacen bien y evitan exponer entidades, mientras que retrofitarlo después es costoso; (2) instalar un `@ControllerAdvice` único antes de escribir el primer controlador, no después, para no repetir la inconsistencia detectada en H09; (3) separar la base de datos por servicio a nivel de *rol y credencial*, no sólo de esquema lógico, porque un `search_path` amplio (como el de H05) anula la separación aunque existan esquemas distintos; (4) definir `deadline` y una política mínima de reintento acotado en el primer cliente gRPC/HTTP que se escriba, porque añadir resiliencia después de tener decenas de llamadas síncronas dispersas, como le ocurre hoy a BCEL, es mucho más caro que diseñarla desde el inicio.

6 Conclusiones y recomendación priorizada

El sistema BCEL demuestra dominio de la arquitectura de microservicios en aspectos que van más allá de lo exigido por el curso: un balanceador con *health checks* activos, un algoritmo propio de elección de líder sobre etcd en `microservicio-soporte`, observabilidad con Prometheus/Grafana ya cableada en `docker-compose.yml`, y un uso correcto y consistente de DTO en `secretaria/soporte`. Estos aciertos deben mantenerse y, en el caso del manejador de excepciones de `secretaria/soporte`, extenderse a `sga-principal`, que es hoy el eslabón más débil del sistema en los cinco bloques evaluados.

La recomendación priorizada para las próximas dos semanas es: primero, rotar y externalizar todos los secretos versionados y revocar el `search_path` cruzado de `docente` (H05, H06, H12 – las

tres severidades críticas, con impacto directo en seguridad y en la integridad de los datos académicos reales del sistema); segundo, añadir **deadline** y degradación a los clientes gRPC (H07/H08), porque sin esto cualquier otra mejora de resiliencia es incompleta; tercero, unificar el manejo de errores (H09) reutilizando el **GlobalExceptionHandler** que ya existe en **secretaria/soporte** como base para **sga-principal**; y en paralelo, dado que no requieren coordinación entre servicios, resolver las violaciones de capas y SOLID de menor riesgo (H01–H04, H10, H13).

Declaración de uso de inteligencia artificial generativa

El análisis del código fuente del repositorio revisado (identificación de violaciones de principios SOLID, patrones y antipatrones, problemas de separación de capas, manejo de excepciones distribuidas y trazabilidad) fue realizado por el equipo AGLS. Se empleó un modelo de lenguaje (Claude, Anthropic) en dos momentos del proceso: primero, para corregir redacción, ortografía, coherencia y claridad de las oraciones del informe una vez redactado el borrador propio; segundo, como verificación cruzada posterior al análisis del equipo: tras completar la revisión manual del código, se le pidió a la IA que analizara de forma independiente el mismo repositorio (con acceso de solo lectura, mediante búsqueda de texto, lectura de archivos y comandos de análisis estático como **grep/wc**, sin modificar el código del equipo BCEL) para contrastar los hallazgos propios y confirmar que no se hubiera dejado de lado ningún aspecto relevante de los cinco bloques de la guía FORM-TL-05. Todos los hallazgos citan evidencia localizada y verificable directamente en el código fuente entregado. El equipo AGLS validó los hallazgos contra un clon real del repositorio, completó los datos de identificación (integrantes, SHA **ff6f8b0**, fecha de auditoría) y abrió los issues correspondientes en el repositorio de BCEL (ver la Nota de verificación en la portada y la Sección 4) .

Referencias

- [1] A. Bacchelli y C. Bird, “Expectations, Outcomes, and Challenges of Modern Code Review,” en *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA: IEEE, 2013, págs. 712-721. DOI: 10.1109/ICSE.2013.6606617.
- [2] C. Sadowski, E. Söderberg, L. Church, M. Sipko y A. Bacchelli, “Modern Code Review: A Case Study at Google,” en *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Gothenburg, Sweden: ACM, 2018, págs. 181-190. DOI: 10.1145/3183519.3183525.
- [3] International Organization for Standardization, “ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model,” ISO, Geneva, Switzerland, inf. téc., 2023.
- [4] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017, pág. 432, ISBN: 978-0-13-449416-6.
- [5] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994, ISBN: 978-0-201-63361-0.
- [6] T. Cerny, A. S. Abdelfattah, A. A. Maruf, A. Janes y D. Taibi, “Catalog and Detection Techniques of Microservice Anti-Patterns and Bad Smells: A Tertiary Study,” *Journal of Systems and Software*, vol. 206, pág. 111 829, 2023. DOI: 10.1016/j.jss.2023.111829.
- [7] R. Tighilt, M. Abdellatif, I. Trabelsi, L. Madern, N. Moha e Y.-G. Guéhéneuc, “On the Maintenance Support for Microservice-Based Systems through the Specification and the Detection of Microservice Antipatterns,” *Journal of Systems and Software*, vol. 204, pág. 111 755, 2023. DOI: 10.1016/j.jss.2023.111755.
- [8] D. Taibi y V. Lenarduzzi, “On the Definition of Microservice Bad Smells,” *IEEE Software*, vol. 35, n.º 3, págs. 56-62, 2018. DOI: 10.1109/MS.2018.2141031.
- [9] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018, ISBN: 978-1-6172-9454-9.
- [10] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.^a ed. Sebastopol, CA, USA: O’Reilly Media, 2021, ISBN: 978-1-4920-3402-5.
- [11] T. I. Mohottige, A. Polyvyanny, C. Fidge, R. Buyya y A. Barros, “Reengineering Software Systems into Microservices: State-of-the-Art and Future Directions,” *Information and Software Technology*, vol. 183, pág. 107 732, 2025. DOI: 10.1016/j.infsof.2025.107732.